CSE1005 — PYTHON PROGRAMMING

Before Loops:

Lists, Tuples, range() & Dictionaries

The building blocks you need before we start for and while loops

School of Computer Science, Engineering & Applications

SESSION ROADMAP

Why Learn These First?

Loops become far more useful once you can store multiple values together. Lists, tuples and dictionaries are Python's core ways to hold collections of data — and range() is what makes for loops work. We'll cover all four today, then next session we loop over them.

01

Lists

Ordered, changeable collections — the most-used data type

02

Tuples

Ordered, unchangeable collections

03

range()

Generating number sequences — essential for loops

04

Dictionaries

Storing data as key–value pairs

01

Lists

Ordered, changeable collections of values

What is a List?

A list is a collection of values stored in a single variable, written with square brackets [ ]. Lists can hold any data type, keep their order, and can be changed after creation.

```
fruits = ["apple", "banana", "cherry"]
marks = [90, 85, 88, 76]
mixed = ["Riya", 21, True]

print(fruits)
print(len(fruits))    # 3 items
```

Key Points

- Written with [ ] square brackets
- Items separated by commas
- Can mix data types
- Keeps insertion order
- len() gives number of items

Accessing Items — Indexing

Each item in a list has a position number called an index. Python starts counting from 0, and negative indexes count from the end.




```
fruits = ["apple", "banana", "cherry", "mango"]
print(fruits[0])      # apple   (first item)
print(fruits[2])      # cherry
print(fruits[-1])     # mango   (last item)
```

Slicing — Getting a Range of Items

A slice returns a sub-list using [start:stop]. The item at stop is not included.

```
nums = [10, 20, 30, 40, 50, 60]

print(nums[1:4])    # [20, 30, 40]  — index 1 up to (not including) 4
print(nums[:3])     # [10, 20, 30]  — from start up to index 3
print(nums[3:])     # [40, 50, 60]  — from index 3 to the end
print(nums[:])      # [10, 20, 30, 40, 50, 60]  — a full copy
```

Common List Methods

append(x)

Add an item to the end

```
fruits.append("kiwi")
```

insert(i, x)

Insert an item at position i

```
fruits.insert(1, "mango")
```

remove(x)

Remove the first matching item

```
fruits.remove("banana")
```

pop(i)

Remove & return item at index i

```
fruits.pop(0)
```

sort()

Sort the list in place

```
marks.sort()
```

len(list)

Number of items in the list

```
len(fruits)
```

Lists are Mutable

Mutable means a list can be changed after it's created — you can update, add or remove items without creating a new list.

```
fruits = ["apple", "banana", "cherry"]

fruits[1] = "blueberry"     # replace an item
print(fruits)                # ['apple', 'blueberry', 'cherry']

fruits.append("mango")
print(fruits)                # ['apple', 'blueberry', 'cherry', 'mango']
```

02

Tuples

Ordered collections that cannot be changed

What is a Tuple?

A tuple is just like a list, but it is written with round brackets ( ) and, once created, it cannot be changed — this is called immutable.

```
point = (2, 3)
colors = ("red", "green", "blue")

print(point[0])      # 2
print(colors[-1])    # blue
```

Key Points

- Written with ( ) round brackets
- Indexing & slicing work the same as lists
- Cannot append, remove or sort
- Used for data that shouldn't change

Tuples Cannot Be Changed

✓ Correct

```
point = (2, 3)
print(point[0])    # 2

# Reading values works fine
```

✕ Incorrect

```
point = (2, 3)
point[0] = 5

# TypeError:
# 'tuple' object does not
# support item assignment
```

List vs Tuple

Aspect	List	Tuple
Brackets	[ ]	( )
Changeable?	Yes (mutable)	No (immutable)
Typical use	Data that will change or grow	Fixed data, e.g. coordinates, RGB colors
Methods available	`append, remove, sort, insert...`	`count(), index() only`
Example	`colors = ["red", "blue"]`	`point = (10, 20)`

03

The range() Function

How Python generates number sequences for loops

What Does range() Do?

range() generates a sequence of numbers. It doesn't create a full list in memory — it produces numbers one at a time, which is exactly what a for loop needs.

`range(5)`	`0, 1, 2, 3, 4`	starts at 0 by default, stops before 5
`range(1, 6)`	`1, 2, 3, 4, 5`	starts at 1, stops before 6
`range(2, 11, 2)`	`2, 4, 6, 8, 10`	start, stop, step of 2
`range(5, 0, -1)`	`5, 4, 3, 2, 1`	counting backwards with a negative step

Turning range() Into a List

You can convert a range into a list using list() to see all the values it produces at once — useful while learning.

```
print(list(range(5)))          # [0, 1, 2, 3, 4]
print(list(range(2, 11, 2)))   # [2, 4, 6, 8, 10]
```

A Sneak Peek at Next Class

```
for i in range(1, 6):
    print(i)      # This is exactly how a for loop will use range()
```

04

Dictionaries

Storing data as key–value pairs

What is a Dictionary?

A dictionary stores data as key–value pairs, written with curly braces { }. Instead of looking up an item by its position (like a list), you look it up by its key.

```
student = {
    "name": "Riya",
    "age": 19,
    "branch": "CSE"
}

print(student["name"])    # Riya
```

Key Points

- Written with { } curly braces
- Each item is a key: value pair
- Keys must be unique
- Access a value using its key, not a number
- Very useful for structured, labeled data

Adding, Updating & Removing Items

Add or update a value by assigning to a key. If the key exists, its value is updated; if not, a new pair is added.

```
student["age"] = 20            # update existing key
student["cgpa"] = 8.7          # add a new key
```

Remove a key–value pair using del or pop().

```
del student["branch"]
student.pop("cgpa")
```

Useful Dictionary Methods

keys()

Returns all the keys

```
student.keys()
```

values()

Returns all the values

```
student.values()
```

items()

Returns key-value pairs together

```
student.items()
```

get(key)

Safely fetch a value (no error if missing)

```
student.get("email")
```

in

Check whether a key exists

```
"name" in student
```

len()

Number of key-value pairs

```
len(student)
```

Quick Recap — Before We Start Loops

1. List [ ] — ordered, changeable collection; access items using an index.
2. Tuple ( ) — ordered, unchangeable collection; use when data shouldn't change.
3. range() — generates a sequence of numbers; the standard tool for for loops.
4. Dictionary { } — stores key–value pairs; access a value using its key.
5. Next class: we will use for and while loops to go through lists, ranges and dictionaries automatically.